Dipartimento di
Ingegneria
e Architettura

Academic Year 2025-26

Mobile Robots

Exploration hackaton

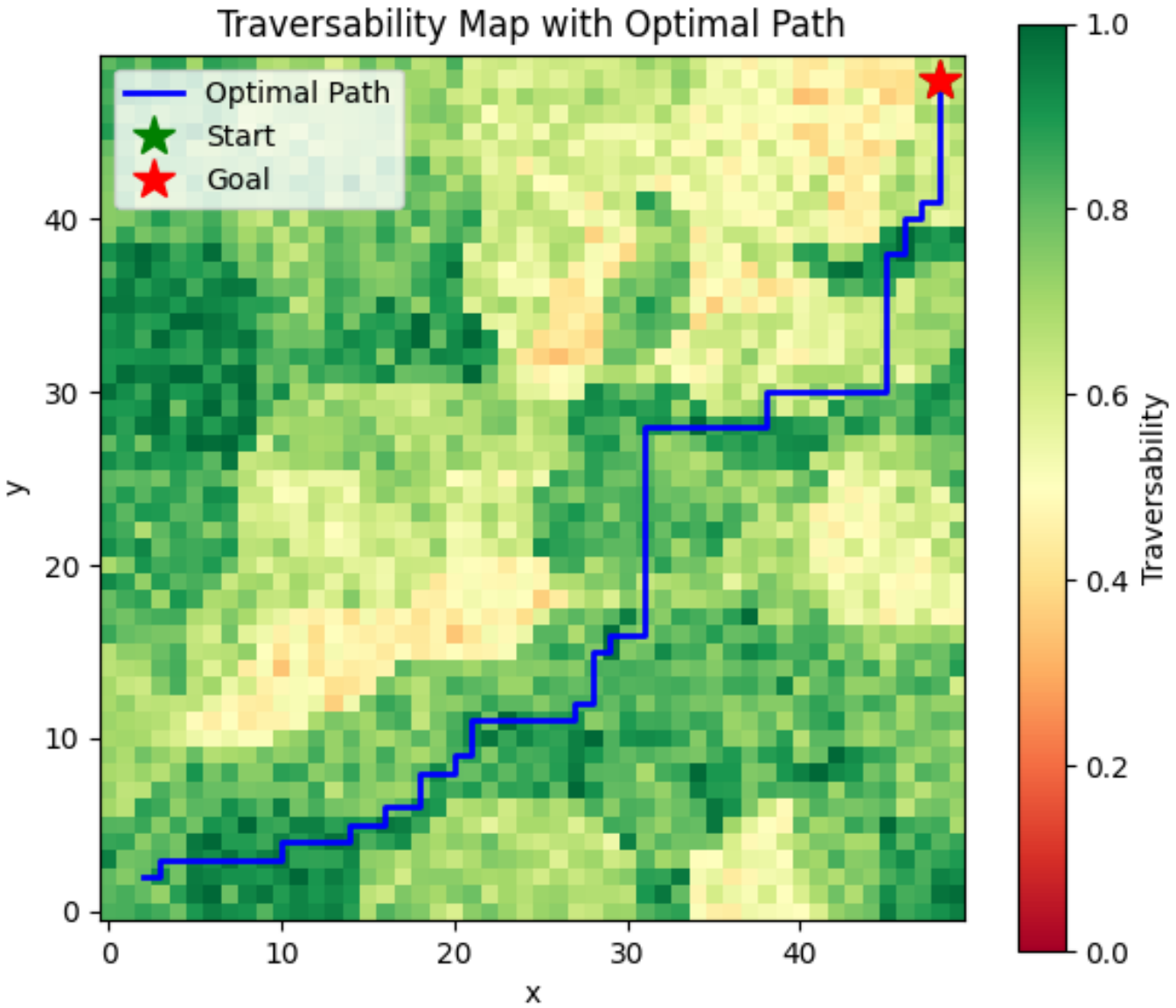
Problem overview

Mars Rover Navigation under Uncertainty

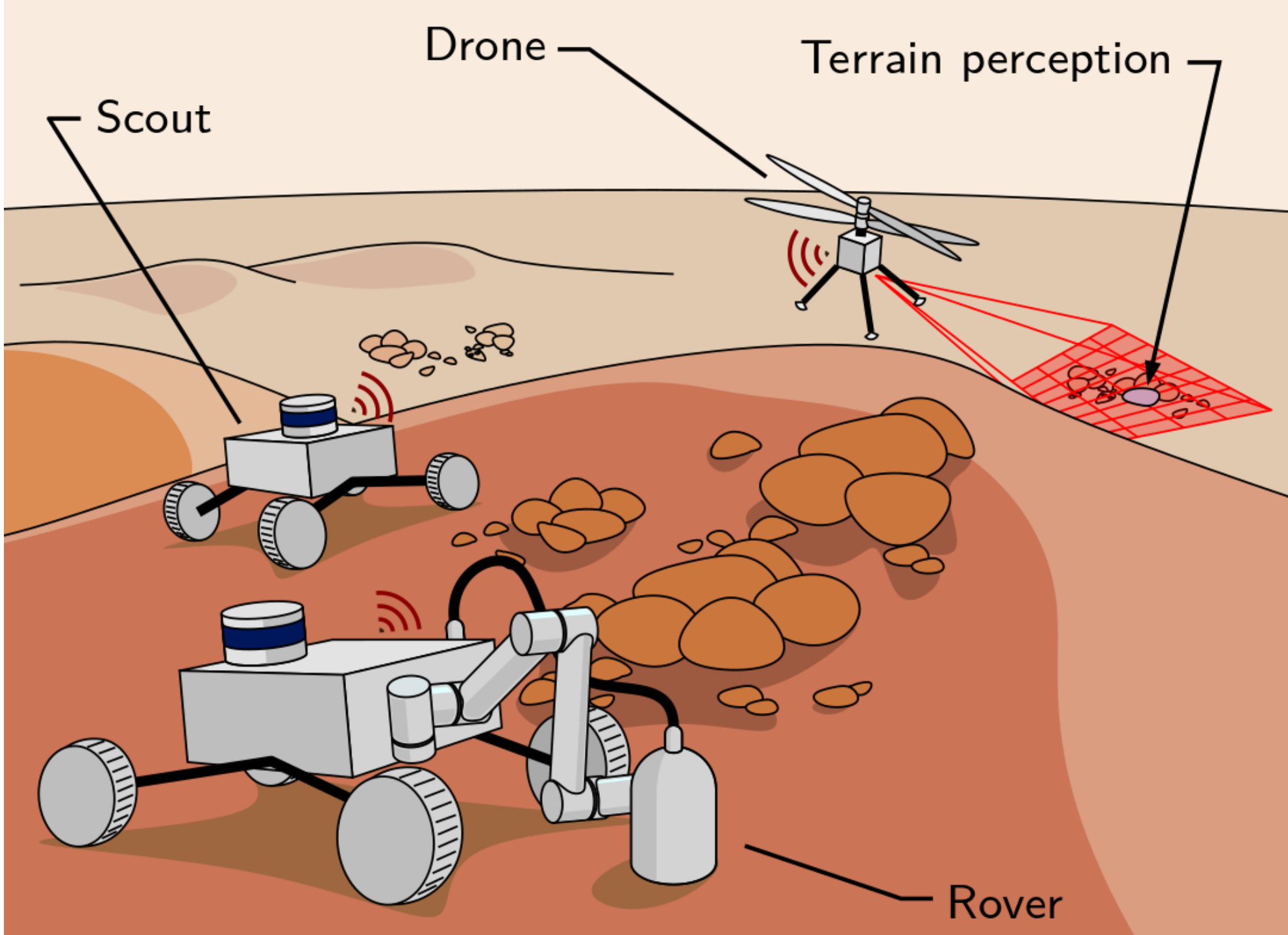
- Navigate a rover from **start → target**
- Terrain is **observable:**
  - **Terrain color** $\in [0,1]$
  - **Terrain texture** $\in [0,1]$
  - Slope $\in [0,30]$ deg
  - Uphill angle $\in [-\pi, +\pi]$ rad
- But its *qualities* you can only estimate:
  - **Traversability**
  - **Stuck events**
- Observables provide **noisy, indirect information**

Core challenge:

- Learn environment properties
- Plan under uncertainty
- Balance **risk vs efficiency**



Scenario



Agents and Roles (I)

<u>You control three agents:</u>

- **Drone**
  - Fast, not affected by terrain
  - Cannot get stuck
  - Needs to **recharge**
  - Provides **local observations (radius-limited)**
- **Scout**
  - Cannot get stuck
  - **Affected by traversability**
  - Used to **probe terrain difficulty**
- **Rover**
  - Must reach the target
  - **Affected by traversability**
  - Can experience **stuck events (penalized)**

<u>Design implication:</u>

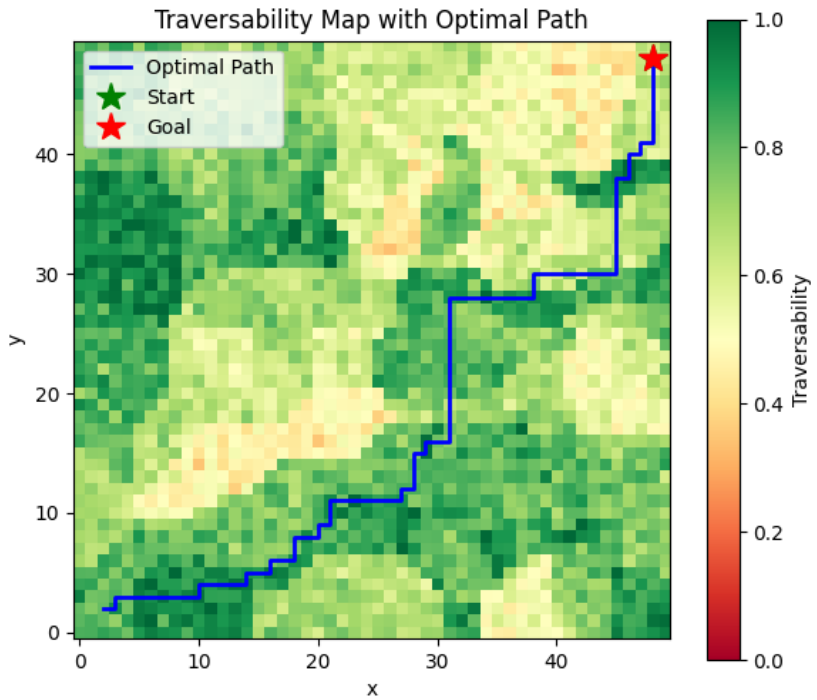
- Drone → perception
- Scout → terrain identification
- Rover → execution

Agents and Roles (II)

- **Drone**
  - Speed **2 m/s**
  - Operations 5 min, recharge 1 hr
  - Unaffected by terrain traversability
  - Cannot get stuck **(no penalty)**
- **Scout**
  - Speed **0.5 m/s**
  - Affected by terrain traversability
  - Cannot get stuck **(no penalty)**
- **Rover**
  - Speed **0.1 m/s**
  - Affected by terrain traversability
  - Can get stuck **(gets penalty)**

Objective

- Create a **governor** that coordinates the agents so that the rover can reach the target location without getting stuck and in the shortest time.



Information structure

- **Available information**
  - Local observables (terrain features)
- **Hidden variables**
  - True traversability
  - True stuck probability (treated at runtime as stuck-events)
- **Statistical structure**
  - Traversability = f(observables) + noise
  - Stuck probability = g(observables) + noise
- **Key property**
  - Prediction accuracy improves with sampling: ~70% → ~99%
- **Implication**
  - This is a **learning + planning problem**, not pure path planning

Scoring function (I)

- A certain number N of runs will be performed (ideally 30, but min 3)
- Each run i is assigned a **cost**:

$$J_i = T_i + \lambda_s \cdot N_i^{stuck} + \lambda_d \cdot D_i^{final}$$

- Where:
  - T_i total mission time
  - N_i^{stuck} number of stuck events
  - D_i^{final} distance to target at end

- Final score: $J = \frac{1}{N} \sum_{i=1}^{N} J_i$

- Objective: **minimize** J (lower is better)

Scoring function (II)

Score coefficients:

- λ_s: tbd
- λ_d: tbd

Maximum time per run:

- T_{max}: tbd (more than enough)

Scoring procedure:

- The map_api.py file will be replaced by one given by the organizers; this doesn't differ in mechanics, but it does rearrange the names of internal variables to make exploits harder
- The teams governor function (`runGovernor(mapApi_instance, startPoint, targetPoint, maximumTime)`) will have to return the following output tuple:

  `(reached_target: boolean, last_distance_from_target: double, time_elapsed: double, stuck_events: double, drone_travel: double, scout_travel: double, rover_travel: double, perceive_calls: int, step_calls: int)`
- The governor function will be run N (tbd) times and results calculated with the scoring function presented in the previous slide.

Interpretation and Trade-offs

The cost captures three competing objectives:

- **Efficiency** → minimize time
- **Robustness** → avoid stuck events
- **Goal achievement** → reach target (or get close)

Trade-offs

- More exploration → safer but slower
- Less exploration → faster but riskier

Good solutions will:

- Use drone for **global awareness**
- Use scout **selectively**
- Plan rover paths with **risk-awareness**

Rule n. 1

- **You are not allowed to exploit the map_api.py class**
  For example:
  - reading properties directly
  - calling private or obfuscated methods

- You can only use the map_api.py class by using the provided public methods:
  - `register_robot()`
  - `step()`
  - `perceive()`

- Refer to hackaton_rules.md for the full list of rules and instructions in docs/

Methods

`__init__(csv_path, config=None, rng_seed=None)`

- **Purpose**
  - Initialize the MapAPI from a CSV terrain file
- **Inputs:**
  - csv_path: Path to terrain data CSV
  - config: MapConfig object (optional, uses defaults)
  - rng_seed: Random seed for reproducibility (optional)
- **Output:**
  - MapAPI instance ready to register robots
- **Throws:**
  - ValueError if CSV invalid
  - FileNotFoundError if missing

Methods

`register_robot(robot_id, robot_type)`

- **Purpose**
  - Add a robot to the simulation with its type
- **Inputs:**
  - robot_id: Unique string identifier
  - robot_type: RobotType.DRONE, SCOUT, or ROVER
    - DRONE: Fast, ignores terrain, never stuck (2 m/s)
    - SCOUT: Affected by terrain, senses stuck (0.5 m/s)
    - ROVER: Terrain+stuck, can immobilize (0.1 m/s)
- **Output:**
  - None
- **Throws:**
  - ValueError if empty ID or already registered

Methods

`step(robot_id, position, command_velocity, command_orientation)`

- **Purpose**
  - Simulate one timestep of robot motion and terrain interaction
- **Inputs:**
  - robot_id: Registered robot identifier
  - position: (x, y) coordinate tuple
  - command_velocity: Desired velocity in m/s
  - command_orientation: Heading angle in radians (0 = +X axis)
- **Output:**
  - StepResult with:
    - actual_velocity: Computed motion after terrain effects
    - is_stuck: Boolean indicating stuck event
    - battery_level:
- **Throws:**
  - KeyError if not registered
  - ValueError if out of bounds
- **Note:**
  - Applies traversability, slopes, stuck logic per robot type

Methods

`perceive(robot_id, position)`

- **Purpose**
  - Get visible terrain observations in 2D radius around robot
- **Inputs:**
  - robot_id: Registered robot identifier
  - position: (x, y) center coordinate for perception
- **Output:**
  - List of TerrainObservation objects with:
    - x, y: Cell coordinates
    - features: Dict with slope, uphill_angle, texture, color
- **Throws:**
  - KeyError if not registered
  - ValueError if radius < 0
- **Notes:**
  - Observation radius set via MapConfig.perceive_radius (default: 2)
  - Uses circular radius (not square)
  - Features include optional noise
  - Excludes hidden fields (traversability, stuck events)